

Git Rebase vs Merge Beginner Guide

Combining code changes safely is one of the most essential daily skills for software engineers. Understanding when to merge and when to rebase keeps your project history clean and prevents team headaches.

What you'll learn:

- 1 Standard Git Merge
- 2 Git Rebase Basics
- 3 Interactive Rebase and Squashing
- 4 Handling Conflicts and Aborting

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Standard Git Merge

Merging takes all commits from your target branch and combines them into your current branch using a new merge commit. This preserves the exact history of how feature branches were created and joined together over time. It is non-destructive, meaning existing commits are never modified or rewritten. This makes it ideal for public branches where keeping an accurate history is important.

```
$git checkout main  
git merge feature-login
```

Cloud & DevOps Daily

Git Rebase vs Merge: When to Use Which (and Why It Matters)

Friday, October 09, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Git Rebase Basics

Rebasing takes your feature commits and replays them one by one on top of the main branch. Imagine cutting your branch's base and pasting it onto the tip of the main branch. This creates a linear, straight-line timeline without extra merge commits cluttering the log. Use this on private local branches before submitting a pull request to keep history clean.

```
$git checkout feature-login  
git rebase main
```

Cloud & DevOps Daily

Git Rebase vs Merge: When to Use Which (and Why It Matters)

Friday, October 09, 2026

Swipe to tip 3 >> [linkedin.com/in/mramanrajsingh](https://www.linkedin.com/in/mramanrajsingh)

3

TIP 3 OF 4

Interactive Rebase and Squashing

Interactive rebase lets you edit, combine, or delete commits before sharing your code with others. Squashing combines several tiny commits into one single meaningful commit message. This is perfect when you made multiple 'WIP' or 'fix typo' commits during development. It cleans up your work so teammates only see one well-organized commit.

```
$git checkout feature-login  
git rebase -i HEAD~3
```

Cloud & DevOps Daily

Git Rebase vs Merge: When to Use Which (and Why It Matters)

Friday, October 09, 2026

Swipe to tip 4 >> [linkedin.com/in/mramanrajsingh](https://www.linkedin.com/in/mramanrajsingh)

4

TIP 4 OF 4

Handling Conflicts and Aborting

When Git encounters overlapping line changes during a rebase, it pauses and asks you to resolve conflicts. You can inspect the files, resolve the conflicting lines, stage them, and continue the rebase. If things get too messy or overwhelming, you can abort the rebase anytime to safely return your branch back to its original state.

```
$git status
# Fix conflict files, then:
git add .
git rebase --continue
# Or to cancel safely:
git rebase --abort
```

Cloud & DevOps Daily

Git Rebase vs Merge: When to Use Which (and Why It Matters)

Friday, October 09, 2026

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always pull the latest main branch changes before starting new work.
- + Use Git Merge when you want an authentic record of branch timelines.
- + Use Git Rebase on local feature branches to maintain a clean linear log.
- + Squash messy intermediate commits before opening a pull request.
- + Test your application thoroughly immediately after any merge or rebase operation.

Watch Out For

- ! Never rebase public shared branches that teammates are actively pulling from.
- ! Using 'git push --force' blindly instead of the safer 'git push --force-with-lease'.
- ! Resolving merge conflicts hastily without checking line-by-line code logic.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh